



数据库系统课程实验报告

实验名称:	数据高级查询
实验日期:	2025.10.21
实验地点:	文宣楼 B308
提交日期:	2025.10.27

学号:	37220232203780
姓名:	马鑫
专业年级:	2023 级数字媒体技术
学年学期:	第三学年第一学期

1.实验目的

- 熟练掌握设计正确的 SQL 查询语句以实现数据高级查询的方法
- 熟练掌握 openGauss 连接查询、子查询和集合查询的语法结构及使用方法
 - （内）连接、（全）外连接、左外连接、右外连接
 - 子查询（嵌套查询）
 - 不相关子查询与相关子查询
 - EXISTS/NOT EXISTS
 - ANY
 - ALL
 - 集合运算：UNION、INTERSECT、MINUS/EXCEPT
- 理解不相关子查询与相关子查询的不同，掌握构造相应 SQL 语句的方法
- 熟练掌握基于派生表的查询方法

2.实验内容和步骤

(1) openGauss 的连接查询语法：

```
SELECT ... FROM table1 [INNER] JOIN table2 ON table1.column=
table2.column;
```

```
SELECT ... FROM table1 FULL [OUTER] JOIN table2 ON
table1.column= table2.column;
```

```
SELECT ... FROM table1 RIGHT [OUTER] JOIN table2 ON
table1.column= table2.column;
```

SELECT ... FROM table1 LEFT [OUTER] JOIN table2 ON

table1.column= table2.column;

创建两张表 palette_a 和 palette_b(结构相同,但表名不同,color 为颜色)

```
CREATE TABLE palette_a (id INT PRIMARY KEY, color VARCHAR2 (100)
NOT NULL);
```

```
CREATE TABLE palette_b (id INT PRIMARY KEY, color VARCHAR2 (100)
NOT NULL);
```

步骤如下:

1.连接数据库

略

2.创建表

```
sales=> CREATE TABLE palette_a
sales-> (
sales(> id INT PRIMARY KEY,
sales(> color VARCHAR2(100) NOT NULL
sales(> );
NOTICE: CREATE TABLE / PRIMARY KEY will create implicit index "palette_a_pkey" for table "palette_a"
CREATE TABLE
sales=> CREATE TABLE palette_b
sales-> (
sales(> id INT PRIMARY KEY,
sales(> color VARCHAR2(100) NOT NULL
sales(> );
NOTICE: CREATE TABLE / PRIMARY KEY will create implicit index "palette_b_pkey" for table "palette_b"
CREATE TABLE
```

(2) 为表 palette_a 添加样例数据: {(1, 'Red'), (2, 'Green'), (3, 'Blue'), (4, 'Purple')}。

步骤如下:

```

sales=> INSERT INTO palette_a(id,color) VALUES(1,'Red');
INSERT 0 1
sales=> INSERT INTO palette_a(id,color) VALUES(2,'Green');
INSERT 0 1
sales=> INSERT INTO palette_a(id,color) VALUES(3,'Blue');
INSERT 0 1
sales=> INSERT INTO palette_a(id,color) VALUES(4,'Purple');
INSERT 0 1

```

(3)为表 palette_b 添加样例数据:{{(1, 'Green'), (2, 'Red'), (3, 'Cyan'), (4, 'Brown')}}。

步骤如下:

```

sales=> INSERT INTO palette_b(id,color) VALUES(1,'Green');
INSERT 0 1
sales=> INSERT INTO palette_b(id,color) VALUES(2,'Red');
INSERT 0 1
sales=> INSERT INTO palette_b(id,color) VALUES(3,'Cyan');
INSERT 0 1
sales=> INSERT INTO palette_b(id,color) VALUES(4,'Brown');
INSERT 0 1

```

(4) 查询两张表中相同颜色的所有信息。

步骤如下:

```

sales=> SELECT * FROM palette_a INNER JOIN palette_b ON palette_a.color=palette_b.color;
 id | color | id | color
----+-----+----+-----
  1 | Red   |  2 | Red
  2 | Green |  1 | Green
(2 rows)

```

(5) 查询 palette_a 表中颜色不出现在 palette_b 表中的 id 和颜色。

步骤如下:

1.

```

sales=> SELECT palette_a.id,palette_a.color FROM palette_a LEFT JOIN palette_b ON palette_a.color=palette_b.color WHERE palette_b.id IS NULL;
 id | color
----+-----
  3 | Blue
  4 | Purple
(2 rows)

```

2.

```
sales=> SELECT id,color FROM palette_a WHERE NOT EXISTS (SELECT 1 FROM palette_b WHERE palette_b.color=palette_a.color);
id | color
----+-----
 3 | Blue
 4 | Purple
(2 rows)
```

3.

```
sales=> SELECT id,color FROM palette_a WHERE color NOT IN (SELECT color FROM palette_b);
id | color
----+-----
 3 | Blue
 4 | Purple
(2 rows)
```

(6) 查询 palette_b 表中颜色不出现在 palette_a 表中的 id 和颜色。

步骤如下：

1.

```
sales=> SELECT b.id,b.color FROM palette_a a RIGHT JOIN palette_b b ON a.color=b.color WHERE a.id IS NULL;
id | color
----+-----
 3 | Cyan
 4 | Brown
(2 rows)
```

2.

```
sales=> SELECT id,color FROM palette_b WHERE NOT EXISTS (SELECT 1 FROM palette_a WHERE palette_a.color=palette_b.color);
id | color
----+-----
 3 | Cyan
 4 | Brown
(2 rows)
```

3.

```
sales=> SELECT id,color FROM palette_b WHERE color NOT IN (SELECT color FROM palette_a);
id | color
----+-----
 3 | Cyan
 4 | Brown
(2 rows)
```

(7) 查询 (5) 或 (6) 两种情况的信息（用（全）外连接）。

步骤如下：

```

sales=> SELECT COALESCE(a.id,b.id) AS id, COALESCE(a.color,b.color) AS color, CASE WHEN a.id IS NULL THEN 'Only in B' WHEN b.id IS NULL THEN 'Only in A' END AS source FROM
palette_a a FULL OUTER JOIN palette_b b ON a.color=b.color WHERE a.id IS NULL OR b.id IS NULL;
id | color | source
-----+-----+-----
3 | Blue | Only in A
4 | Purple | Only in A
3 | Cyan | Only in B
4 | Brown | Only in B
(4 rows)

```

(8) 查询产品表 products 中的 product_id, product_name, list_price 信息, 要求产品定价 list_price 大于其平均定价 list_price。

步骤如下:

```

sales=> SELECT product_id, product_name, list_price FROM products WHERE list_price > (SELECT AVG(list_price) FROM products);
product_id | product_name | list_price
-----+-----+-----
228 | Intel Xeon E5-2699 V3 (OEM/Tray) | 341046.00
248 | Intel Xeon E5-2697 V3 | 277498.00
249 | Intel Xeon E5-2698 V3 (OEM/Tray) | 266072.00
2 | Intel Xeon E5-2697 V4 | 255499.00
45 | Intel Xeon E5-2685 V3 (OEM/Tray) | 250169.00
46 | Intel Xeon E5-2695 V3 (OEM/Tray) | 243195.00
47 | Intel Xeon E5-2697 V2 | 237709.00
51 | Intel Xeon E5-2695 V4 | 226999.00
91 | Intel Xeon E5-2695 V2 | 225999.00
93 | Intel Xeon E5-2690 (OEM/Tray) | 211672.00
98 | Intel Xeon E5-2687W V3 | 206499.00
102 | Intel Xeon E5-2687W V4 | 204269.00
158 | Intel Xeon E5-2667 V3 (OEM/Tray) | 200946.00
159 | Intel Xeon E5-2690 V4 | 199449.00
160 | Intel Xeon E5-2690 V3 | 190873.00
163 | Intel Xeon E5-2683 V4 | 189999.00
169 | Intel Xeon E5-2683 V4 (OEM/Tray) | 184489.00
240 | Intel Core i7-4960X Extreme Edition | 180597.00
242 | Intel Xeon E5-1680 V3 (OEM/Tray) | 175199.00
243 | Intel Xeon E5-2643 V4 (OEM/Tray) | 170886.00
19 | Intel Core i7-6950X (OEM/Tray) | 170437.00
52 | Intel Xeon E5-2670 V3 | 167698.00
165 | Intel Xeon E5-2680 | 166661.00
212 | Intel Xeon E5-2680 V4 | 163999.00
166 | Intel Xeon E5-2680 V3 (OEM/Tray) | 163889.00
82 | Intel Core i7-6950X | 149989.00
213 | Intel Xeon E5-2643 V3 (OEM/Tray) | 146996.00
218 | Intel Xeon E5-2660 V4 | 138889.00
219 | Intel Xeon E5-2660 V3 | 129973.00
85 | Intel Xeon E5-2660 V2 (OEM/Tray) | 127400.00

```

(9) 查询产品表 products 中最便宜产品的 product_id, product_name, list_price。

步骤如下:

```

sales=> SELECT product_id, product_name, list_price FROM products WHERE list_price = (SELECT MIN(list_price) FROM products);
product_id | product_name | list_price
-----+-----+-----
286 | Samsung MZ-V6E500 | 234.00
(1 row)

```

(10) 查询没有一个订单的顾客姓名。

步骤如下:

1.

```
sales=> SELECT name FROM customers WHERE customer_id NOT IN (SELECT DISTINCT customer_id FROM orders);
      name
-----
United Continental Holdings
INTL FCStone
Publix Super Markets
ConocoPhillips
3M
Exelon
Tesoro
Northwestern Mutual
Enterprise Products Partners
Rite Aid
Qualcomm
EMC
Time Warner Cable
Northrop Grumman
Lear
Genuine Parts
Omnicom Group
Monsanto
National Oilwell Varco
Marriott International
Kinder Morgan
Molina Healthcare
Lincoln National
CH Robinson Worldwide
Synnex
HollyFrontier
PBF Energy
Waste Management
Parker-Hannifin
```

2.

```
sales=> SELECT name FROM customers WHERE NOT EXISTS (SELECT 1 FROM orders WHERE orders.customer_id=customers.customer_id);
      name
-----
United Continental Holdings
INTL FCStone
Publix Super Markets
ConocoPhillips
```

3.

```
sales=> SELECT name FROM customers LEFT JOIN orders ON customers.customer_id
=orders.customer_id WHERE orders.order_id IS NULL;
      name
-----
United Continental Holdings
INTL FCStone
Publix Super Markets
ConocoPhillips
```

4.


```

sales=> SELECT name FROM customers c LEFT JOIN orders o ON c.customer_id=o.customer_id GROUP BY c.customer_id,c.name HAVING COUNT(o.order_id)=0;
name
-----
American Airlines Group
Toys R Us
AIG
Gilead Sciences
CVS Health

```

(11) 查询产品表 products 中产品的 product_id, product_name, list_price, 要求产品定价 list_price 大于其同类产品（可由 category_id 表达）的平均定价。

步骤如下:

1.

```

sales=> SELECT product_id,product_name,list_price FROM products p1 WHERE list_price>(SELECT AVG(list_price) FROM products p2 WHERE p1.category_id=p2.category_id);
product_id | product_name | list_price
-----
228 | Intel Xeon E5-2699 V3 (OEM/Tray) | 341046.00
248 | Intel Xeon E5-2697 V3 | 277498.00
249 | Intel Xeon E5-2698 V3 (OEM/Tray) | 266072.00
2 | Intel Xeon E5-2697 V4 | 255499.00
45 | Intel Xeon E5-2685 V3 (OEM/Tray) | 250169.00
46 | Intel Xeon E5-2695 V3 (OEM/Tray) | 243195.00
47 | Intel Xeon E5-2697 V2 | 237709.00
51 | Intel Xeon E5-2695 V4 | 226999.00
91 | Intel Xeon E5-2695 V2 | 225999.00
93 | Intel Xeon E5-2690 (OEM/Tray) | 211672.00
98 | Intel Xeon E5-2687W V3 | 206499.00
102 | Intel Xeon E5-2687W V4 | 204269.00
158 | Intel Xeon E5-2667 V3 (OEM/Tray) | 200946.00
159 | Intel Xeon E5-2690 V4 | 199449.00
160 | Intel Xeon E5-2690 V3 | 190873.00
163 | Intel Xeon E5-2683 V4 | 189999.00
169 | Intel Xeon E5-2683 V4 (OEM/Tray) | 184489.00
240 | Intel Core i7-4960X Extreme Edition | 180597.00
242 | Intel Xeon E5-1680 V3 (OEM/Tray) | 175199.00
243 | Intel Xeon E5-2643 V4 (OEM/Tray) | 170886.00
19 | Intel Core i7-6950X (OEM/Tray) | 170437.00
52 | Intel Xeon E5-2670 V3 | 167698.00
165 | Intel Xeon E5-2680 | 166661.00
212 | Intel Xeon E5-2680 V4 | 163999.00
166 | Intel Xeon E5-2680 V3 (OEM/Tray) | 163889.00
82 | Intel Core i7-6950X | 149989.00
213 | Intel Xeon E5-2643 V3 (OEM/Tray) | 146996.00
218 | Intel Xeon E5-2660 V4 | 138889.00
219 | Intel Xeon E5-2660 V3 | 129973.00
85 | Intel Xeon E5-2660 V3 (OEM/Tray) | 127499.00

```

2.

```

sales=> SELECT product_id,product_name,list_price FROM (SELECT *,AVG(list_price) OVER (PARTITION BY category_id) AS avg_category_price FROM products) AS category_avg WHERE list_price>avg_category_price;
product_id | product_name | list_price
-----
228 | Intel Xeon E5-2699 V3 (OEM/Tray) | 341046.00
248 | Intel Xeon E5-2697 V3 | 277498.00
249 | Intel Xeon E5-2698 V3 (OEM/Tray) | 266072.00
2 | Intel Xeon E5-2697 V4 | 255499.00

```

3.

```

sales=> SELECT p.product_id,p.product_name,p.list_price FROM products p JOIN (SELECT category_id,AVG(list_price) AS avg_price FROM products GROUP BY category_id) AS category_avg ON p.category_id=category_avg.category_id WHERE p.list_price>category_avg.avg_price;
product_id | product_name | list_price
-----
228 | Intel Xeon E5-2699 V3 (OEM/Tray) | 341046.00
248 | Intel Xeon E5-2697 V3 | 277498.00
249 | Intel Xeon E5-2698 V3 (OEM/Tray) | 266072.00
2 | Intel Xeon E5-2697 V4 | 255499.00

```

(12) 查询有订单 order 的所有顾客 customer 姓名（查询涉及 customers 表和 orders 表）。

步骤如下:

1.

```
sales=> SELECT name FROM customers c WHERE EXISTS (SELECT 1 FROM orders o WHERE o.customer_id=c.customer_id);
      name
-----
Facebook
Supervalu
NextEra Energy
PG&E Corp.
Goodyear Tire & Rubber
Micron Technology
ConAgra Foods
Bank of New York Mellon Corp.
Sempra Energy
Raytheon
Plains GP Holdings
US Foods Holding
AbbVie
Centene
Community Health Systems
Alcoa
International Paper
Emerson Electric
Aflac
AutoNation
Progressive
Abbott Laboratories
Dollar General
Tenet Healthcare
Eli Lilly
AECOM
Jabil Circuit
CenturyLink
General Mills
```

2.

```
sales=> SELECT name FROM customers WHERE customer_id IN (SELECT DISTINCT customer_id FROM orders);
      name
-----
Facebook
Supervalu
NextEra Energy
PG&E Corp.
```

3.

```
sales=> SELECT DISTINCT c.name FROM customers c INNER JOIN orders o ON c.customer_id=o.customer_id;
      name
-----
Sempra Energy
Dollar Tree
Nucor
Thermo Fisher Scientific
```

4.

```

sales=> SELECT c.name FROM customers c JOIN orders o ON c.customer_id=o.customer_id GROUP BY c.customer_id,c.name;
      name
-----
Progressive
Thermo Fisher Scientific
General Mills
Community Health Systems

```

(13) 执行以下三条与 NULL 相关的语句，观察各自执行的结果，能否从中得出某些初步结论？

```
SELECT * FROM customers WHERE customer_id IN (NULL);
```

```
SELECT NULL FROM customers;
```

```
SELECT * FROM customers WHERE EXISTS (SELECT NULL FROM
customers);
```

步骤如下：

1. 运行 3 条语句

```

sales=> SELECT * FROM customers WHERE customer_id IN (NULL);
customer_id | name | address | website | credit_limit
-----+-----+-----+-----+-----
(0 rows)

```

```

sales=> SELECT NULL FROM customers;
?column?
-----

```

```
(319 rows)
sales=> SELECT * FROM customers WHERE EXISTS (SELECT NULL FROM customers);
customer_id | name | address | website | credit_limit
-----
177 | United Continental Holdings | 2904 S Salina St, Syracuse, NY | http://www.unitedcontinentalholdings.com | 5000.00
180 | INTL FCStone | 5344 Haverford Ave, Philadelphia, PA | http://www.intlfcstone.com | 5000.00
184 | Publix Super Markets | 1795 Wu Meng, Muang Chonburi, | http://www.publix.com | 1200.00
187 | ConocoPhillips | Walpurgisstr 69, Munich, | http://www.conocophillips.com | 2400.00
190 | 3M | Via Frenzy 6903, Roma, | http://www.3m.com | 1200.00
192 | Exelon | Via Luminosa 162, Firenze, | http://www.exeloncorp.com | 500.00
199 | Tesoro | 6700 W. 13th St, Denver, CO | http://www.tesoro.com | 3600.00
165 | Johnson Controls | 6Th And Master St, Philadelphia, PA | http://www.johnsoncontrols.com | 3600.00
170 | Goldman Sachs Group | 23985 Bedford Rd N, Battle Creek, MI | http://www.gs.com | 3700.00
174 | Oracle | 701 Seneca St, Buffalo, NY | http://www.oracle.com | 5000.00
(319 rows)
```

2. 得出结论

NULL 与任何值的比较都返回 UNKNOWN，而查询只返回为 TRUE 的值；NULL 可以作为有效的查询结果；EXISTS 只关心行是否存在，不关心具体值。

(14) 找出所有没有订单的顾客姓名（查询涉及 customers 表和 orders 表）。

步骤如下：

1.

```
sales=> SELECT c.name FROM customers c WHERE NOT EXISTS (SELECT 1 FROM orders o WHERE o.customer_id=c.customer_id);
name
-----
United Continental Holdings
INTL FCStone
Publix Super Markets
ConocoPhillips
3M
Exelon
Tesoro
Northwestern Mutual
Enterprise Products Partners
Rite Aid
Qualcomm
EMC
Time Warner Cable
Northrop Grumman
Lear
Genuine Parts
Omnicom Group
Monsanto
National Oilwell Varco
Marriott International
Kinder Morgan
Molina Healthcare
Lincoln National
CH Robinson Worldwide
Synnex
HollyFrontier
PBF Energy
```

2.

```
sales=> SELECT name FROM customers WHERE customer_id NOT IN (SELECT DISTINCT customer_id FROM orders);
name
-----
United Continental Holdings
INTL FCStone
Publix Super Markets
ConocoPhillips
```

3.

```
sales=> SELECT c.name FROM customers c LEFT JOIN orders o ON c.customer_id=o.customer_id WHERE o.order_id IS NULL;
name
-----
United Continental Holdings
INTL FCStone
Publix Super Markets
ConocoPhillips
```

4.

```
sales=> SELECT c.name FROM customers c LEFT JOIN orders o ON c.customer_id=o.customer_id GROUP BY c.customer_id,c.name HAVING COUNT(o.order_id)=0;
name
-----
American Airlines Group
Toys R Us
AIG
Gilead Sciences
```

(15) 查询产品表 products 中的产品名 product_name 和定价 list_price, 要求其定价高于产品种类 1 中的任何产品定价。

步骤如下:

1.

```
sales=> SELECT product_name,list_price FROM products WHERE list_price>ANY (SELECT list_price FROM products WHERE category_id=1);
product_name | list_price
-----+-----
Intel Xeon E5-2699 V3 (OEM/Tray) | 341046.00
Intel Xeon E5-2697 V3 | 277498.00
Intel Xeon E5-2698 V3 (OEM/Tray) | 266072.00
Intel Xeon E5-2697 V4 | 255499.00
Intel Xeon E5-2685 V3 (OEM/Tray) | 250169.00
Intel Xeon E5-2695 V3 (OEM/Tray) | 243195.00
Intel Xeon E5-2697 V2 | 237709.00
Intel Xeon E5-2695 V4 | 226999.00
Intel Xeon E5-2695 V2 | 225999.00
Intel Xeon E5-2643 V2 (OEM/Tray) | 2200.00
Intel Xeon E5-2690 (OEM/Tray) | 211672.00
Intel Xeon E5-2687W V3 | 206499.00
Intel Xeon E5-2687W V4 | 204269.00
Intel Xeon E5-2667 V3 (OEM/Tray) | 200946.00
Intel Xeon E5-2690 V4 | 199449.00
Intel Xeon E5-2690 V3 | 190873.00
Intel Xeon E5-2470V2 | 19047.00
Intel Xeon E5-2683 V4 | 189999.00
Intel Xeon E5-2637 V2 (OEM/Tray) | 1850.00
Intel Xeon E5-2683 V4 (OEM/Tray) | 184489.00
Intel Core i7-4960X Extreme Edition | 180597.00
Intel Xeon E5-2699 V4 (OEM/Tray) | 1756.00
Intel Xeon E5-1680 V3 (OEM/Tray) | 175199.00
Intel Xeon E5-2643 V4 (OEM/Tray) | 170886.00
Intel Core i7-6950X (OEM/Tray) | 170437.00
Intel Xeon E5-2670 V3 | 167698.00
Intel Xeon E5-2680 | 166661.00
Intel Xeon E5-2680 V4 | 163999.00
```

2.

```

sales=> SELECT product_name,list_price FROM products WHERE list_price>(SELECT MIN(list_price) FROM products WHERE category_id=1);
      product_name      | list_price
-----+-----
Intel Xeon E5-2699 V3 (OEM/Tray) | 341046.00
Intel Xeon E5-2697 V3      | 277498.00
Intel Xeon E5-2698 V3 (OEM/Tray) | 266072.00
Intel Xeon E5-2697 V4      | 255499.00

```

3.

```

sales=> SELECT p1.product_name,p1.list_price FROM products p1 WHERE EXISTS (SELECT 1 FROM products p2 WHERE p2.category_id=1 AND p1.list_price>p2.list_price);
      product_name      | list_price
-----+-----
Intel Xeon E5-2699 V3 (OEM/Tray) | 341046.00
Intel Xeon E5-2697 V3      | 277498.00
Intel Xeon E5-2698 V3 (OEM/Tray) | 266072.00
Intel Xeon E5-2697 V4      | 255499.00

```

(16) 查询产品表 products 中的产品名 product_name 和定价 list_price, 要求其定价高于产品种类 1 中的所有定价。

步骤如下:

1.

```

sales=> SELECT product_name,list_price FROM products WHERE list_price>ALL (SELECT list_price FROM products WHERE category_id=1);
      product_name      | list_price
-----+-----
PNY VCQP6000-PB          | 549999.00
Intel SSDPECM040T401     | 886799.00
(2 rows)

```

2.

```

sales=> SELECT product_name,list_price FROM products WHERE list_price>(SELECT MAX(list_price) FROM products WHERE category_id=1);
      product_name      | list_price
-----+-----
PNY VCQP6000-PB          | 549999.00
Intel SSDPECM040T401     | 886799.00
(2 rows)

```

3.

```

sales=> SELECT p1.product_name,p1.list_price FROM products p1 WHERE NOT EXISTS (SELECT 1 FROM products p2 WHERE p2.category_id=1 AND p1.list_price<=p2.list_price);
      product_name      | list_price
-----+-----
PNY VCQP6000-PB          | 549999.00
Intel SSDPECM040T401     | 886799.00
(2 rows)

```

(17) 查询产品表 products 中的产品名 product_name 和定价 list_price, 要求其定价低于产品种类的所有平均定价。

步骤如下:

1.

```

sales-> SELECT product_name,list_price FROM products WHERE list_price<ALL (SELECT AVG(list_price) FROM products GROUP BY category_id);

```

product_name	list_price
Intel Xeon E5-2643 V2 (OEM/Tray)	2200.00
Intel Xeon E5-2470V2	19047.00
Intel Xeon E5-2637 V2 (OEM/Tray)	1850.00
Intel Xeon E5-2699 V4 (OEM/Tray)	1756.00
Intel Xeon E5-2650 V2	1249.00
Intel Core 2 Extreme QX9775	892.00
Intel Core i7-4770K	799.00
Intel Core i7-3930K	660.00
PNY VCQM6000-24GB-PB	4139.00
ATI FirePro S9050	1699.00
AMD FirePro S7000	12185.00
NVIDIA VCQM4000-PB	790.00
Crucial	799.00
Kingston	6535.00
GSkill Ripjaws V Series	6452.00
Kingston	644.00
Asus X99-E-10G WS	649.00
Asus PRIME X299-DELUXE	4873.00
Asus X99-DELUXE/U31	4403.00
ASRock EP2C612 WS	35849.00
ASRock Z270 SuperCarrier	35398.00
Asus MAXIMUS VIII EXTREME/ASSEMBLY	35398.00
Asus STRIX X299-E GAMING	34999.00
Gigabyte X299 AORUS Ultra Gaming	34399.00
Asus TUF X299 MARK 1	33999.00
Asus Z170-WS	33899.00
MSI X299 GAMING PRO CARBON AC	33781.00
MSI X99A XPOWER GAMING TITANIUM	32999.00

2.

```

sales-> SELECT product_name,list_price FROM products WHERE list_price<(SELECT MIN(avg_price) FROM (SELECT AVG(list_price) AS avg_price FROM products GROUP BY category_id) AS category_avg);

```

product_name	list_price
Intel Xeon E5-2643 V2 (OEM/Tray)	2200.00
Intel Xeon E5-2470V2	19047.00
Intel Xeon E5-2637 V2 (OEM/Tray)	1850.00
Intel Xeon E5-2699 V4 (OEM/Tray)	1756.00

3.

```

sales-> SELECT p1.product_name,p1.list_price FROM products p1 WHERE NOT EXISTS (SELECT 1 FROM (SELECT category_id,AVG(list_price) AS avg_price FROM products GROUP BY category_id) AS ca WHERE p1.list_price=>ca.avg_price);

```

product_name	list_price
Intel Xeon E5-2643 V2 (OEM/Tray)	2200.00
Intel Xeon E5-2470V2	19047.00
Intel Xeon E5-2637 V2 (OEM/Tray)	1850.00
Intel Xeon E5-2699 V4 (OEM/Tray)	1756.00

(18) 查询 contacts 表和 employees 表中的所有 last_name, 并以 last_name 升序显示。

步骤如下:

1.

```

sales=> SELECT last_name FROM contacts UNION SELECT last_name FROM employees ORDER BY last_name ASC;
last_name
-----
Abbott
Alexander
Allison
Alston
Arnold
Atkinson
Avila
Bailey
Baldwin
Ball
Barnes
Barnett
Barrera
Barry
Battle
Bauer
Beard
Beasley
Bell
Bennett
Benson
Benton
Black
Blair
Booker
Bowen
Boyd

```

2.

```

sales=> SELECT DISTINCT last_name FROM (SELECT last_name FROM contacts UNION ALL SELECT last_name FROM employees) AS combined_names ORDER BY last_name ASC;
last_name
-----
Abbott
Alexander
Allison
Alston

```

3.

```

sales=> SELECT last_name FROM (SELECT last_name,ROW_NUMBER() OVER (PARTITION BY last_name ORDER BY last_name) as rn FROM (SELECT last_name FROM contacts UNION ALL SELECT last_name FROM employees) AS all_names) AS ranked_names WHERE rn=1 ORDER BY last_name ASC;
last_name
-----
Abbott
Alexander
Allison
Alston
Arnold

```

(19) 查询 contacts 表和 employees 表中的所有 last_name, 并以 last_name 升序显示。

步骤如下:


```
sales=> SELECT last_name FROM contacts UNION ALL SELECT last_name FROM employees ORDER BY last_name ASC;
last_name
-----
Abbott
Alexander
Allison
Alston
Arnold
Atkinson
Atkinson
Avila
Bailey
Baldwin
Ball
Barnes
Barnett
Barnett
Barrera
Barry
Battle
Bauer
Beard
Beasley
Bell
Bennett
Benson
Benton
Black
Blair
Booker
```

(20) 查询同时出现在 contacts 表和 employees 表中的所有 last_name。

步骤如下：

1.

```

sales=> SELECT last_name FROM contacts INTERSECT SELECT last_name FROM employees;
last_name
-----
Stone
Webb
Henry
Brooks
Flores
Cruz
Simmons
Mason
Cole
Murray
Wallace
Ford
Butler
Woods
Sanders
Myers
Robertson
Ferguson
Grant
Henderson
West
Rose
Hayes
Ortiz
Jordan
Nichols
Bryant
Spencer
McDonald
(29 rows)

```

2.

```

sales=> SELECT DISTINCT c.last_name FROM contacts c INNER JOIN employees e ON c.last_name=e.last_name;
last_name
-----
Stone
Webb
Henry
Brooks

```

3.

```

sales=> SELECT DISTINCT c.last_name FROM contacts c WHERE EXISTS (SELECT 1 FROM employees e WHERE e.last_name=c.last_name);
last_name
-----
Stone
Webb
Henry
Brooks

```

4.

```

sales=> SELECT DISTINCT last_name FROM contacts WHERE last_name IN (SELECT last_name FROM employees);
last_name
-----
Stone
Webb
Henry
Brooks

```

(21) 查询在产品表 products 中而不在库存表 inventories 中的产品号 product_id。

步骤如下：

1.

```

sales=> SELECT product_id FROM products EXCEPT SELECT product_id FROM inventories;
product_id
-----
267
60
1
246
143
82
164
52
179
208
55
183
178
268
169
219
64
206
48
180
202
156
77

```

2.

```

sales=> SELECT p.product_id FROM products p WHERE NOT EXISTS( SELECT 1 FROM inventories i WHERE i.product_id=p.product_id);
product_id
-----
164
162
92
1

```

3.

```

sales=> SELECT p.product_id FROM products p LEFT JOIN inventories i ON p.product_id=i.product_id WHERE i.product_id IS NULL;
product_id
-----
      164
      162
       92
        1

```

4.

```

sales=> SELECT product_id FROM products WHERE product_id NOT IN (SELECT DISTINCT product_id FROM inventories);
product_id
-----
      248
      249
       45
       51

```

3.实验总结

3.1 完成的工作

在前两次实验的基础上，完成本次实验要求的查询操作。

3.2 对实验的认识

本次实验运用了大量高级查询的操作，增加了对 EXIST/NOT EXIST, ANY, ALL, 连接查询，子查询，集合查询等的熟练程度。

思考题回答如下：

- 请分析归纳只能使用子查询的查询要求特点并举例说明。

(1) 存在性检查相关记录是否存在，而不关心具体值。比如查询有订单的用户 SELECT name FROM customers c WHERE EXISTS (SELECT 1 FROM orders o WHERE o.customer_id=c.customer_id);。

(2) 相关子查询依赖外层查询的值。比如 (11) 查询产品定价高于其同类产品的平均定价。

(3) 集合比较需要与子查询的结果集进行比较。比如 (17) 查询定价低于产品种类的所有平均定价的产品信息。

(4) 需要在分组列上显示聚合查询的结果。比如查询每个产品其同类型的平均定价 SELECT (SELECT AVG(list_price) FROM

```
products p2 WHERE p2.product_id=p1.product_id) as avg_list_price  
FROM products p1;。
```

- 相关子查询与不相关子查询的区别？试举例说明哪些情形下使用相关子查询，哪些情形下相关子查询可替换为一般查询实现？

不相关子查询不依赖外层查询，相关子查询依赖外层查询的值。

同上一个思考题的第（1）点，对同一个表进行存在性检查时，必须使用相关子查询；比如（12），可以使用连接查询来替换子查询。

3.3 遇到的困难及解决方法

无。